Safety-Aware Mutation Testing

User guide to the current prototype: assessment, decisions, reporting, and limitations.

> Safety review is a risk-identification aid. It is not a sandbox or a guarantee that running a project, its tests, or a mutant is safe. Run trusted projects and commands in an appropriately isolated environment.

1. What the feature does

Mutation testing introduces a proposed change to source code, runs the existing tests, and records whether the tests detect the change. Some mutations can alter conditions or calls that influence database operations, file writes, network requests, processes, or loop termination. Safety awareness reviews each proposed mutant before it is written and tested.

The local analyzer examines supported syntax and recognized operation names. It can identify a possible link between a changed condition and a later or nested operation. Its explanation describes evidence of a potential risk, not proof of what will execute.

2. Where safety review fits in a run

1. Baseline

The runner creates a project working copy and runs the configured baseline compile/test commands. Optional coverage commands may also run. These steps happen before the per-mutant safety gate and can have side effects.

2. Mutation proposal

The selected MSL specification identifies a mutation point. The runner reads the source and creates the proposed mutated source in memory. Mutants on uncovered lines can be marked NO_COVERAGE without being executed.

3. Assessment

The safety controller compares the original and proposed source, selects a local analyzer for the language, and produces a level, category, reason, and assessor. If the mutation cannot be applied at its recorded location, it is reported as a compile error.

4. Gate

SAFE proceeds automatically. REVIEW and UNKNOWN request a user decision in an interactive terminal. BLOCK cannot be continued. The separate gate enforces these choices.

5. Test and restore

When allowed, the runner writes the mutation to its working copy and runs the configured compile/test commands. It records the result and attempts to restore the source afterward. Skipped mutants are not executed.

6. Reporting

The console and HTML report include safety assessments and outcomes. Safety-skipped and aborted mutants are excluded from the score, since they were not tested.

3. Assessment levels and choices

Level	Meaning	Interactive choices
SAFE	Narrow recognized low-risk case	Runs automatically
REVIEW	A possible risk has been identified	Continue once, skip, or abort
UNKNOWN	Effects cannot be established	Continue once, skip, or abort
BLOCK	A recognized destructive change	Skip or abort; continue is unavailable

Interactive choices are c to continue once, s to skip, and a to abort the run. Invalid choices are rejected. If input closes or cannot be read, the mutant is skipped. Decisions are not saved between runs.

4. Worked example: database guard

Suppose an application checks user input and returns early when it is invalid, then commits a database change later in the same function:

```
if not data or 'title' not in data:
    return error_response()
# ...
db.session.commit()
```

A specification may replace the condition with False. The Python analyzer can identify the changed if condition, the early exit, and the later db.session.commit call. It can produce REVIEW / GUARD_CLAUSE_DATABASE: removing the guard might change whether the commit is reached.

This is a potential control-flow relationship. The analyzer does not prove that the commit runs, that the call name refers to a particular runtime object, or that a real database will be changed. A user should inspect the mutation, the commands, and the environment before approving it.

5. Language coverage and local analysis

ANTLR syntax-tree analyzers support branch-condition checks in Python, Java, JavaScript, C#, and Racket. Selected loops are also recognized for Python, Java, JavaScript, and C#. Racket loop forms and direct-call changes outside Python are not yet analyzed structurally.

Python can inspect changes inside certain direct calls and trace possible same-file chains through up to four uniquely named functions. Python and JavaScript can identify selected changed early-return guards before a later recognized operation; JavaScript also follows some possible same-file helper-name paths. These matches are syntactic and can be imprecise. Aliases, imports, shadowing, reassignment, dispatch, duplicate names, and recursive cycles are not fully resolved.

The fallback contextual assessor uses limited text patterns, including a rule for newly introduced explicit destructive commands. This is not a comprehensive database, file, or process policy. Unsupported constructs, parse errors, and unresolved cases can produce UNKNOWN.

6. Reading the results

The assessment describes a possible safety impact before execution. The mutation result describes what happened after a mutant was allowed to run. KILLED means the configured tests failed; SURVIVED means they passed. SAFETY_SKIPPED and SAFETY_ABORTED identify mutants that were not executed due to a safety decision.

Example: 13 generated mutants, 3 killed, 7 survived, and 3 safety-skipped yield a score of 3 / (3 + 7) = 30%. The skipped mutants are excluded. A mutation score is not a security score, and its interpretation depends on the selected rules, tests, coverage, and skipped mutants.

7. Unattended and CI execution

Without an interactive terminal, REVIEW, UNKNOWN, and BLOCK are skipped by default. A configuration may set unattended_safety=run_unknown to test UNKNOWN mutants automatically; REVIEW and BLOCK remain skipped. The provided GitHub Actions template sets this option. Use it only when the execution environment and consequences are understood.

```
unattended_safety=run_unknown
```

If all mutants are skipped, the run has no tested mutants from which to assess the test suite. Consult the counts and skipped reasons rather than treating a zero score as evidence about test quality.

8. Optional model review

Model review is disabled by default. With explicit opt-in and valid credentials and model configuration, an optional OpenAI reviewer can provide a second opinion for eligible local REVIEW or UNKNOWN results. Local SAFE and BLOCK assessments bypass it. The model cannot authorize execution or override a block; a local REVIEW is not downgraded to SAFE.

For each eligible request, the tool sends the full original and mutated source file, its filename, and compile/test command strings. It does not send other project files or test contents. Requests can incur charges and increase runtime. Files over 30,000 characters are not sent. Share code only when authorized.

In PowerShell, configure environment variables for the current terminal if remote review is intentionally desired:

```
$env:OPENAI_API_KEY = "<your key>"
$env:MUTATION_SAFETY_MODEL = "<supported model>"
$env:MUTATION_SAFETY_SEND_CODE = "yes"
```

Disable opt-in by removing MUTATION_SAFETY_SEND_CODE from the environment. Do not put credentials in project files or Git. Missing settings, refusals, malformed responses, timeouts, or service failures fall back to the local assessment or UNKNOWN. The project documentation reports simulated response tests, not a live-account validation.

9. Limitations and safe operating practices

The reviewer examines limited source context and may miss aliases, indirect calls, code in other files, generated code, malicious behavior, and effects in build or test commands. It may also warn about harmless operations. SAFE is not a guarantee and BLOCK covers only certain recognized patterns.

A project copy and source restoration do not prevent a command from modifying an external database, contacting a service, or changing files outside the copy. An abrupt termination can prevent cleanup. Use disposable credentials and resources where appropriate, review configured commands, and use isolation suitable for the project.

> Current status: implemented prototype. The client has not yet confirmed the proposed safety design. This guide documents the current implementation, not a claim of complete risk detection.

10. Further information

See SAFETY.md in the repository for supported patterns, safety tests, optional model review, and the included sample-project demonstration. See docs/ci/README.md for CI behavior and docs/spec-format.md for mutation specification syntax.